DDD

DDD 구조(비즈니스 로직 중심)

Controller → Application → Domain → Infrastructure

DDD 구성요소

- [ ] Domain
  - [ ] 비즈니스 규칙이 들어가는 곳
  - [ ] 순수 파이썬 코드
- [ ] Application(유스케이스)
  - [ ] 로직 흐름 담당
  - [ ] 계산 및 검증 단계는 도메인에 맡김
- [ ] Infrastructure(Repository / 외부 통신)
  - [ ] DB, API 연결 담당
    - [ ] Domain 객체로 변환해서 넘김
- [ ] Controller(Presentation)
  - [ ] FastAPI와 같은 API 계층
  - [ ]

개발

- [ ] Domain 레이어
  - [ ] User / Account
  - [ ] 이메일 형식/ 비밀번호 규칙
  - [ ] 탈퇴 불가 조건?
  - [ ] 동일 이메일 중복 가입 방지 규칙
  - [ ] JWT vs 세션 중 뭘 쓸지에 따라 Infrastructure 구조가 완전히 달라짐

	JWT	세션
저장소	클라이언트	서버 (Redis 등)
로그아웃 구현	복잡 (블랙리스트 필요)	단순 (세션 삭제)
수평 확장	유리	Redis 필요

JWT 쓴다면 추가 고려:

- Access Token + Refresh Token 분리할지
- Refresh Token을 DB에 저장할지 (탈취 대응)
- 로그아웃 시 블랙리스트 어디에 저장할지 (Redis?)

- [ ] Application 레이어
  - [ ] `SignUpUseCase` — 이메일 인증 포함할지?
  - [ ] `LoginUseCase` — 소셜 로그인(OAuth) 나중에 붙일 가능성?
  - [ ] `LogoutUseCase` — 단일 기기 로그아웃 vs 전체 기기 로그아웃
  - [ ] `WithdrawUseCase` — 즉시 삭제 vs 30일 유예 후 삭제
- [ ] Infrastructure 설계
- [ ] 보안 설계
  - [ ] 비밀번호 해싱 어느 레이어에서 할지(Domain or Application)
  - [ ] 로그인 실패 횟수 제한 어디서 관리
  - [ ] 탈퇴 후 데이터 보존 정책
  - [ ] 토큰 만료 시간 정책

ㄹㅇ 설계 구조

- [ ] Domain 레이어
  - [ ] User Entity

```
User
├── id: UserId            ← DB auto-increment PK
├── email: Email          ← 로그인 식별자
├── password: HashedPassword
├── is_active: bool       ← 탈퇴 시 False
```

```
├── is_verified: bool    ← 이메일 인증 여부
├── deleted_at: datetime | None    ← 30일 유예 기준
├── created_at: datetime
└── updated_at: datetime
```

☐ Valude Object

Value Object	담당 검증
`UserId`	양의 정수 / Auto Increment
`Email`	형식 검증 (RFC 표준), 소문자 정규화 / @ 필수로 들어가야함
`RawPassword`	8-16자, 특수문자 포함 여부 / DB 저장 암호화
`HashedPassword`	해시된 문자열 보관, 비교 메서드 포함 /
`RefreshTokenId`	UUID 형식 여부

- ~~아직 RefreshToken 왜 사용하는지 모르겟음~~

```
1. 왜 Acces Token 하나로는 안되는가?
    유효기간 짧게(30분) -> 30분마나 로그인 해야함
    유효기간 길게(30일) -> 탈취당하면 길게는 30일동안 위험함
    따라서 둘로 나눔
    1) Acces Token 유효기간 짧게 (30분~1시간)
            실제 API 호출에도 사용
            탈취당해도 금방 만료 피해 적음
    2) Refresh Token 유효기간 길게(7~30일)
            Acces Token 재발급 용도로 사용
            API 호출엔 사용 불가
-----
실제 흐름
로그인
  → Access Token (1시간) + Refresh Token (7일) 발급

1시간 후 Access Token 만료
  → Refresh Token으로 새 Access Token 요청
  → 재로그인 없이 자동 갱신

7일 후 Refresh Token도 만료
  → 그제서야 재로그인
```

Refresh Token 저장 위치별 장단점:

위치	장점	단점
DB (권장)	강제 로그아웃 가능, 기기별 관리 가능, 장애에도 안전	매번 DB 조회 발생
Redis	빠름	Redis 장애 시 전체 로그아웃, 비용 추가
쿠키만	DB 저장 불필요	강제 로그아웃 불가, 탈취 위험

→ 저희 설계에선 **DB 저장**으로 갈게요. 단일 기기 로그아웃을 하려면 기기별로 어떤 토큰이 있는지 서버가 알아야 하거든요.

Refresh 탈취 루트

```
1. 네트워크 탈취
   → HTTPS 안 쓰면 전송 중 가로채기 가능
   → HTTPS 쓰면 막힘

2. XSS (Cross-Site Scripting)
   → 토큰을 localStorage에 저장하면
   → 악성 자바스크립트가 그대로 읽어서 가져감

3. 기기 탈취
   → 폰/컴퓨터 자체를 탈취당하면 저장된 토큰도 같이

4. 서버 DB 해킹
   → Refresh Token을 DB에 평문으로 저장했을 때
   → DB 털리면 모든 유저 토큰 노출

----
따라서
쿠키에 저장 시:
  → HttpOnly 설정  : JS가 읽지 못하게 (XSS 방어)
  → Secure 설정    : HTTPS에서만 전송
  → SameSite 설정  : 다른 사이트에서 요청 못하게

DB 저장 시:
  → 평문 저장 금지
  → 해시해서 저장 (탈취당해도 원본 모름)
```

```
Refresh Token Rotation:
  → 재발급할 때마다 Refresh Token도 새로 발급
  → 이전 토큰은 즉시 폐기
  → 탈취당한 토큰 써도 이미 무효화되어 있음
```

- ~~블랙리스트의 정확한 의미를 모르겠음~~

```
정상적인 상황:
로그인 → Access Token 발급 (유효기간 30분)
→ 이 토큰 가진 사람은 30분간 API 자유롭게 사용 가능

문제 상황:
유저가 로그아웃 했어
→ 근데 누군가 그 토큰을 중간에 훔쳐갔다면?
→ 서버는 "이 토큰 유효하네" 하고 그냥 통과시킴
→ 남은 유효시간 동안 그 사람이 내 계정으로 API 호출 가능

해결책 = 블랙리스트:
로그아웃 할 때 → "이 토큰 폐기" 목록에 등록
→ 이후 요청에서 폐기 목록에 있으면 → 가게 진상손님처럼 입장 거부

JWT 토큰 안에는 jti라는 토큰 고유번호가 있다.
이 번호를 블랙리스트에 등록해
토큰 전체를 저장하는 게 아니라 번호표만 저장하는 형태
```

- [ ] UseCase

`SignUpUseCase`

```
1. Email Valud Object 생성 -> 형식 검증
2. Email 중복 확인
3. Password DB에서 가져와서 해싱 -> HashedPassword에 저장
4. User.is_verified = False 선언
5. 이메일 인증 토큰 생성 -> MailPort 발송
6. DB 저장
```

`EmailVerifyUseCase`

```
1. 토큰 유효성 확인(SignUpUseCase 5번)
2. User.is_verified = True 업데이트
3. 인증 토큰 삭제
```

`LoginUseCase`

```
1. Email로 User 조회
2. is_active,is_verified 체크(is_active는 User 생성할때 선언됨)
3. deleted_at 체크 (회원 탈퇴 유예기간인지)
4. 비밀번호 비교 (HashPassword.verify)
5. Acces Token + Refresh Token 발급
6. Refresh Token DB 저장
```

`LogoutUseCase`

```
1. 요청의 Refresh Token ID로 해당 row 찾기
2. Acces Token Jti -> Redis 블랙리스트 등록 (블랙리스트가 뭐임?)
3. Refresh Token row 삭제
```

`WithdrawUseCase`

```
1. User.is_active = False
2. User.deleted_at = 현재 시각
3. 해당 유저의 모든 Refresh Token 삭제(전체 로그아웃)
4. 30일 후 스케줄러가 실제 데이터 처리
```

- [ ] 해싱 위치
  - [ ] 인터페이스로 추상화(추상화 까진 이해했지만 구조는 뭔지 모르겠음

```
Domain / Application
  └── CryptoPort (인터페이스)
        ├── hash(plain: str) → str
        └── verify(plain: str, hashed: str) → bool

Infrastructure
```

```
└── BcryptCryptoAdapter (구현체)
      → CryptoPort를 implement
```

- 이렇게 할때에 이점
  - Domain이 bcrypt를 직접 import 안해도 됨 (bcrypt는 외부 라이브러리 (Infrastructure 영역) → Domain이 Infrstructure의 영역을 사용하면 원칙 위반
  - 나중에 argon2로 교체해도 구현체만 갈아끼우면 됨(argon2 뭔지 이해 안감)
  - 이메일 인증 토큰 암호화, 기타 민감정보 암호화도 같은 포트 활용 가능

- Infrastructure

컴포넌트	역할
`UserRepository`	User CRUD, 이메일 중복 조회
`RefreshTokenRepository`	기기별 Refresh Token 관리
`EmailVerifyTokenRepository`	이메일 인증 토큰 저장/조회/삭제
`BlacklistRepository`	Redis — jti 블랙리스트 / 로그아웃시 jti 저장
`RateLimitRepository`	Redis — IP별 요청 횟수
`BcryptCryptoAdapter`	CryptoPort 구현
`MailAdapter`	인증메일 발송 (SMTP or SES 등)

- IP Rate Limit

```
정상적인 로그인 시도:
  틀려도 몇 번 다시 시도하겠지

해킹 시도 (브루트포스):
  1초에 비밀번호 수천 개 자동으로 시도
  → 언젠간 맞는 비밀번호 찾아냄

Rate Limit:
  같은 IP에서 1분에 10번 이상 로그인 시도
  → "잠깐, 10분 후에 다시 오세요" 차단
```

- 탈퇴

```
WithdrawUseCase 실행 시 즉시:
  is_active  = False
  deleted_at = 현재 시각
  email      = "deleted_1234@unknown.com"  ← 익명화
               (숫자는 UserId)
  password   = "(탈퇴처리됨)"              ← 의미없는 값으로
덮어쓰기

30일 후 스케줄러:
  민감 개인정보 추가 익명화 처리
  (이름, 연락처 등 추후 필드 추가 시)
  실제 Row 삭제는 하지 않음
```

☐ Exception 구조

```
BaseAppException (최상위 커스텀 예외)
  │
  ├── DomainException (도메인 규칙 위반)
  │     ├── InvalidEmailFormatException
  │     ├── InvalidPasswordFormatException
  │     └── InvalidValueObjectException
  │
  ├── ApplicationException (유스케이스 흐름 오류)
  │     ├── AuthenticationException
  │     │     ├── InvalidCredentialsException  "이메일 또
는 비밀번호가 틀렸습니다"
  │     │     ├── AccountInactiveException     "비활성화된
계정입니다"
  │     │     ├── EmailNotVerifiedException    "이메일 인
증이 필요합니다"
  │     │     └── TokenExpiredException        "토큰이 만
료되었습니다"
  │     │
  │     ├── SignUpException
  │     │     └── DuplicateEmailException      "이미 사용
중인 이메일입니다"
  │     │
  │     ├── TokenException
```

```
  |     |     ├── InvalidTokenException        "유효하지
않은 토큰입니다"
  |     |     ├── TokenBlacklistedException    "폐기된 토
큰입니다"
  |     |     └── RefreshTokenNotFoundException "토큰을
찾을 수 없습니다"
  |     |
  |     └── WithdrawException
  |           └── AlreadyWithdrawnException    "이미 탈퇴
한 계정입니다"
  |
  └── InfrastructureException (외부 연동 오류)
        ├── DatabaseException
        ├── RedisException
        └── MailSendException
```

전체 구조

Controller
└── /auth/signup, /login, /logout, /withdraw, /verify-email

Application (UseCase)
└── SignUp / EmailVerify / Login / Logout / Withdraw

Domain
└── User (Entity)
└── Email, HashedPassword, UserId, RefreshTokenId (Value Object)
└── CryptoPort, UserRepositoryPort, MailPort (인터페이스)

Infrastructure
└── UserRepository, RefreshTokenRepository
└── BlacklistRepository (Redis)
└── RateLimitRepository (Redis)
└── EmailVerifyTokenRepository (Redis)
└── BcryptCryptoAdapter, MailAdapter